

Pointcept 训练/测试管线现状 与 DoE 七因子可行性详解

TU Berlin · MDT 研究组 · Yucan Luo · 2026-06-20

0 写作目的与结论先行

任务书 (Aufgabenstellung) 原始增强策略定义了 7 个处理参数: scale、spatial_shape、step、jitter_range、voxel_size、drop_prob、align_mode。本文档逐一对照 Pointcept 框架 (本课题实际使用的训练/测试代码库) 的源码, 回答一个具体问题: 这 7 个参数里, 哪些可以像两周 DoE 计划 (v2) 里的 grid_size / point_max / jitter_sigma / dropout_ratio 那样, "改配置文件里的数字就能跑", 哪些不能、为什么不能、要补多少代码才能。

✓ 结论: scale 部分可用 (机制不同), voxel_size / drop_prob / jitter_range 经改名后可直接复用现成 Transform 类 (即 v2 计划已采用的 grid_size / dropout_ratio / jitter_sigma)。spatial_shape、step、align_mode 三个因子在当前训练/测试管线中完全没有对应的可配置代码路径, 需要先新增 Python 实现, 才能纳入 DoE 矩阵。

1 Pointcept 训练 / 测试管线总览

Pointcept 用统一的 Transform 注册机制 (@TRANSFORMS.register_module()) 把数据预处理拆成一串可在配置文件里拼装的步骤。"一个参数能否做 DoE" 等价于 "这个参数是不是某个已注册 Transform 类的构造函数参数"。训练和测试走的是两条不同的数据流。

1.1 训练时数据流 (逐 epoch 重复)

```
raw .npy point cloud (coord / color / segment ...)
|
v
Dataset.get_data(idx)
|
v
self.transform = Compose([...]) # train.transform in config
|-- RandomScale(scale=[0.95, 1.05])
|-- RandomJitter(sigma=..., clip=...)
|-- GridSample(grid_size=...)
|-- SphereCrop(point_max=...)
|-- RandomDropout(dropout_ratio=...)
`-- ToTensor / Collect ...
|
v
collate_fn -> model.forward() -> loss -> backward
```

图1: 训练时单个样本的处理流程。

- Dataset.get_data(idx) —— 从磁盘读取一个场景 (或已离线分块好的场景)
- self.transform = Compose([...]) —— 配置文件 train.transform 列表逐项执行:
 - RandomScale(scale=[0.95,1.05]) ← "scale" 在这里, 但只是小幅扰动, 不是绝对缩放
 - RandomJitter(sigma=..., clip=...) ← "jitter_range" 对应这里
 - GridSample(grid_size=...) ← "voxel_size" 对应这里
 - SphereCrop(point_max=...) ← 按点数裁剪, 不是按物理尺寸裁剪
 - RandomDropout(dropout_ratio=...) ← "drop_prob" 对应这里

- 每一行"|--"都是配置文件 transform 列表里的一个已注册 Transform 类实例，构造参数即可直接在 .py 配置文件中改数值。

1.2 测试时数据流（推理一个完整场景）

```

Dataset.prepare_test_data(idx)          # defaults.py:159
|
v
for aug in self.aug_transform:          # TTA: random flip / rotate
    data_dict_list.append(aug(...))
    |
    v
for data in data_dict_list:
    data_part_list = self.test_voxelize(data)    # grid hash
    for data_part in data_part_list:
        data_part = self.test_crop(data_part)    # point-count crop
        fragment_list += data_part
    |
    v
SemSegTester.test()                    # test.py:122
    pred = zeros((N, num_classes))
    for fragment in fragment_list:
        pred_part = model(fragment)["seg_logits"]
        pred[idx_part] += softmax(pred_part)
    pred.argmax(dim=1)  ->  final label

```

图2：测试时单个场景的处理流程。

- for aug in self.aug_transform —— TTA：多次不同的随机翻转/旋转，每次都是"整个场景"的一个增强版本
- self.test_voxelize(data) —— 体素化（grid hash），可能因哈希冲突产生多份
- self.test_crop(data_part) —— 按点数裁剪成多个 fragment
- pred[idx_part] += softmax(pred_part) —— 同一个点出现在多个 fragment 里就累加概率
- 注意 fragment_list 的来源是 "TTA 增强 × 体素化哈希"，不是 "按固定 step 切出的、有明确重叠带的空间网格"。这一点是 align_mode 缺失的根源，第4.3节详细展开。

2 已存在、可直接 DoE 的因子 ✓ 可直接 DoE

以下因子在图1的训练流程中已经是某个 Transform 类的构造参数，改配置文件数值即可生效，不需要写任何新代码。这正是 v2 两周计划选用它们的原因。

任务书因子	v2 计划字段	对应类 / 源码位置	构造参数 → 配置写法
voxel_size	grid_size	GridSample transform.py:840	GridSample(grid_size=0.02)
drop_prob	dropout_ratio	RandomDropout transform.py:218	RandomDropout(dropout_ratio=0.1)
jitter_range	jitter_sigma	RandomJitter transform.py:358	RandomJitter(sigma=0.05, clip=0.2)
(块大小的替代)	point_max	SphereCrop transform.py:1015	SphereCrop(point_max=80000)

源码：RandomJitter（与 jitter_range 机制一致，单位/参数名不同）

```
class RandomJitter(object):
    def __init__(self, sigma=0.01, clip=0.05):
        self.sigma = sigma
        self.clip = clip
    def __call__(self, data_dict):
        jitter = np.clip(self.sigma * np.random.randn(N, 3), -self.clip, self.clip)
        data_dict["coord"] += jitter
        return data_dict
```

源码：RandomDropout（与 drop_prob 机制一致）

```
class RandomDropout(object):
    def __init__(self, dropout_ratio=0.2, dropout_application_ratio=0.5):
        self.dropout_ratio = dropout_ratio
    def __call__(self, data_dict):
        if random.random() < self.dropout_application_ratio:
            idx = np.random.choice(n, int(n * (1 - self.dropout_ratio)), replace=False)
            data_dict = index_operator(data_dict, idx)
        return data_dict
```

★ GridSample.grid_size 和 SphereCrop.point_max 的代码见第1.1节图1及下文第3章对照表，不再重复贴出。这四个因子的共同特征：构造函数参数 = 配置文件字段 = DoE 因子，三者是同一个东西，没有额外的"翻译"工作。

3 七因子状态总表

汇总任务书 7 个因子在 Pointcept 管线中的现状、差异、以及若要补全所需的工程量估计（按一人全职投入估算，不含调试与返工时间）。

因子	现状	差异说明	补全工程量估计
scale 坐标缩放因子	部分有	RandomScale 范围仅 [0.95,1.05] 的局部抖动，不是 20 – 100 倍绝对坐标归一化；需要新写一个绝对缩放 transform	~0.5 天（新 Transform 类 + 单测）
spatial_shape 空间裁剪尺寸	无	SphereCrop 按点数裁剪成不规则点簇，无按物理边长裁剪立方体的逻辑	~1 天（BoxCrop 类 + 验证形状）
step 滑动窗口步长	无（仅离线）	chunk_stride 只存在于离线脚本 sampling_chunking_data.py，训练管线不会调用它，每改一次 step 需重新生成整份数据集	~2 – 3 天（改造为在线可调 + 数据管理）
voxel_size 体素降采样尺寸	有	GridSample.grid_size，机制完全一致	0 天（已可用，即 v2 的 grid_size）
drop_prob 点丢弃概率	有	RandomDropout.dropout_ratio，机制完全一致	0 天（已可用）
jitter_range 平移抖动范围	有	RandomJitter.sigma/clip，机制一致，参数形式不同	0 天（已可用，即 v2 的 jitter_sigma）
align_mode 边界对齐模式	无	test.py 现有的 fragment 累加是 TTA 投票，不是按 step 切块后的重叠区域一致性融合；没有"对齐/不对齐"开关	~2 天（独立推理脚本 + 两种融合逻辑）

★ 合计：若要把 7 因子原样搬进 PB 矩阵，预计需要 5 – 6 天纯工程实现（不含训练耗时），这还没算上"在 S3DIS 这种本来就不需要切块的数据集上验证这些参数是否有意义"这个更根本的问题（见第 6 章）。

4 三个缺失因子详解 管线缺失

这一章是全文核心：逐一展示"现有最接近的代码"、"和任务书要求的差距具体在哪一行"、"如果要补，需要新增什么"。

4.1 spatial_shape —— 没有按物理尺寸裁剪的逻辑

任务书定义：spatial_shape 是块裁剪尺寸，取值范围 $[256^3] \rightarrow [1024^3]$ ，单位是体素/物理尺寸的立方体边长。

Pointcept 训练管线里负责"把大场景切小"的类是 SphereCrop：

```
class SphereCrop(object):
    def __init__(self, point_max=80000, sample_rate=None, mode="random"):
        self.point_max = point_max
    def __call__(self, data_dict):
        if data_dict["coord"].shape[0] > point_max:
            center = data_dict["coord"][np.random.randint(N)] # pick a random center point
            idx_crop = np.argsort(np.sum(np.square(coord - center), 1))[:point_max]
            data_dict = index_operator(data_dict, idx_crop) # keep nearest N points to it
        return data_dict
```

关键差异：SphereCrop 控制的是点数（point_max），输出是一个以随机点为中心、KNN 意义上"最近的 N 个点"组成的不规则点簇——不是一个边长固定、轴对齐的立方体。配置文件里没有任何字段叫 spatial_shape，因为没有类去读取这样的字段。

要补什么

- 新写一个 BoxCrop 类，按 x/y/z 物理范围而非点数过滤点：`mask = (coord >= corner) & (coord < corner + spatial_shape)`
- 需要决定裁剪中心的选取策略（随机 / 滑窗起点），并处理裁剪后点数为 0 或极少的边界情况
- 注册进 TRANSFORMS，再在配置文件里替换/追加到 transform 列表中

4.2 step —— 存在，但活在训练管线之外的离线脚本里

任务书定义：step 是滑动窗口步长，控制块之间的重叠率，范围 16 – 128。Pointcept 确实有一个"切块"脚本，但它是预处理阶段一次性运行的命令行工具，不是训练时动态生效的 transform：

```
# pointcept/datasets/preprocessing/sampling_chunking_data.py
def chunking_scene(name, dataset_root, split, grid_size=None,
                  chunk_range=(6, 6), chunk_stride=(3, 3), chunk_minimum_size=10000):
    ...
    for chunk in chunks:
        # grid origins spaced by chunk_stride
        mask = (coord[:,0] >= chunk[0]) & (coord[:,0] < chunk[0]+chunk_range[0]) & ...
        np.save(chunk_save_path / f"{key}.npy", data_dict[key][mask]) # writes new directory on disk

# CLI usage (one-shot, run before training starts):
# python sampling_chunking_data.py --dataset_root xxx --chunk_stride 3 3 --chunk_range 6 6
```

关键差异：这个 chunking_scene() 函数不会被 tools/train.py 调用。它的输出是磁盘上一份新目录（如 train_grid20mm_chunk6x6_stride3x3/），训练配置文件的数据.train.split 字段需要手动改去指向这个新目录。也就是说，"改一个 step 数值"实际等于"重跑一次预处理脚本 + 生成一份新数据集 + 改配置文件路径"，而不是"改配置文件里的一个数字"。

要补什么

- 把 chunk_stride 的切分逻辑改写成一个在线 Transform（训练时实时切，而非提前生成新目录），或者写一个小的批处理脚本自动为 PB 矩阵里每个 step 取值生成对应数据集目录并管理命名

- 需要额外的磁盘空间预算：每个 step 取值对应一份完整的重新分块数据集
- S3DIS 单房间点数本身不大，要先确认 chunk_minimum_size 过滤后是否还有足够多的块可用于训练（否则该因子在小数据集上可能退化成本部一样）

4.3 align_mode —— 现有的"多预测累加"是 TTA 投票，不是边界融合开关

任务书定义：align_mode 是滑窗推理时，重叠区域是否做一致性融合的二元开关（对齐 / 非对齐）。Pointcept 测试时确实会对同一个点的多次预测做累加，容易被误认为已经实现了这个功能：

```
# pointcept/engines/test.py:185-204 (SemSegTester.test)
pred = torch.zeros((segment.size, num_classes)).cuda()
for i in range(len(fragment_list)):
    pred_part = self.model(input_dict)["seg_logits"]
    pred_part = F.softmax(pred_part, -1)
    for be in input_dict["offset"]:
        pred[idx_part[bs:be], :] += pred_part[bs:be]    # accumulate prob if point is in multiple fragments
pred = pred.max(1)[1]    # final argmax
```

问题在 fragment_list 从哪来。看 defaults.py 的 prepare_test_data：

```
# pointcept/datasets/defaults.py:169-185
for aug in self.aug_transform:    # multiple TTA: random flip/rotate, each a whole-scene copy
    data_dict_list.append(deepcopy(data_dict))
for data in data_dict_list:
    data_part_list = self.test_voxelize(data)    # voxel hashing, may yield several copies
    for data_part in data_part_list:
        data_part = self.test_crop(data_part)    # point-count crop (same SphereCrop logic)
        fragment_list += data_part
```

关键差异：fragment_list 的"重叠"来自 TTA 随机增强和体素哈希，每个 fragment 仍然是覆盖（接近）整个场景的一次预测，多份预测累加本质是"对同一场景投票"，回答的问题是"做几次随机增强能不能让预测更稳"。任务书要的 align_mode 回答的是另一个问题："把场景按固定 step 切成有重叠带的网格之后，重叠带的点该怎么处理"——这是空间意义上的边界缝合，现有代码里没有任何分支在处理"两个相邻网格块的重叠区域"这件事，因为现有代码根本不知道哪两个 fragment 是"空间相邻"的。

要补什么（v2 计划第3.3节已经规划了这部分，结论一致）

- 需要一个独立的推理脚本：按 step/overlap ratio 显式切出有重叠带的空间网格（不依赖 TTA）
- 记录每个网格块的空间范围，推理后能判断"这个点落在几个块里"
- 实现条件 A（重叠区域多块 logits 平均）和条件 B（重叠区域取最后写入值）两条路径并对比

5 scale 因子：量级不匹配，不是完全没有

scale 比较特殊：Pointcept 里确实有同名机制，但作用量级和任务书要求差两个数量级，直接拿来用会让"scale 因子"名不副实。

```
class RandomScale(object):
    def __init__(self, scale=None, anisotropic=False):
        self.scale = scale if scale is not None else [0.95, 1.05]    # default: +/-5% jitter
    def __call__(self, data_dict):
        scale = np.random.uniform(self.scale[0], self.scale[1], 3 if self.anisotropic else 1)
        data_dict["coord"] *= scale
        return data_dict
```

任务书的 scale (20 – 100) 是"绝对坐标缩放因子"，用于把原始坐标（米）映射到一个更适合体素化/网络输入的数值范围，是预处理阶段对整个场景做一次性归一化；而 RandomScale 是训练时每个 batch 都重新随机采样的小幅几何增强（±5%），目的是防止模型过拟合固定尺度，两者要解决的问题完全不同。把 RandomScale 的范围直接改成 [20,100] 会让场景坐标在每个 epoch 剧烈抖动，而不是任务书设想的"用一个固定值重新标定坐标系"。

要补什么

- 新写一个一次性的 AbsoluteScale（或在数据预处理阶段加一步），按固定 scale 值缩放坐标，区别于训练时的随机增强 RandomScale

6 即使补完代码，S3DIS 也未必是测这三个因子的好场景

spatial_shape / step / align_mode 这三个参数设计的初衷，是解决"一个场景太大、GPU 显存装不下、必须切块处理"的问题。S3DIS 每个 Area 约 10^6 点（任务书原文数据），当前基线训练显存峰值仅 ~8 – 14 GB（见两周DoE计划第1节基线数据），在 A6000 48GB 上本来就能整场景塞进显存，不依赖切块就能训练。

与之相对，任务书第一章明确列出真正需要切块策略的场景：ArCH（500万 – 1500万点/场景）、SensatUrban（30亿点）、DALES（5亿点）。在"不需要切块就能跑"的数据集上测"切块参数"，观测到的效应很可能是噪声或退化成无效因子（因为不管 step/spatial_shape 怎么变，反正整个场景都能进显存，结果不会有系统性差异）——这是比"代码缺失"更根本的有效性问题的。

建议：spatial_shape / step / align_mode 这套因子，应该留到任务书第五 – 八阶段（在 ArCH / DALES 等真正大场景上）做，而不是现在用 S3DIS 凑数据。

7 结论与建议

结论

1. voxel_size、drop_prob、jitter_range 三个因子已经是 Pointcept 现成 Transform 的构造参数，零额外开发成本，这也是 v2 计划选用对应字段（grid_size / dropout_ratio / jitter_sigma）的原因。
2. scale 在 Pointcept 中存在同名但量级完全不同的机制（ $\pm 5\%$ 训练抖动 vs 20 – 100 倍绝对归一化），直接复用会曲解这个因子的物理含义。
3. spatial_shape（立方体裁剪）、step（在线滑窗步长）、align_mode（边界融合开关）在当前训练/测试管线中没有任何可配置的代码路径——分别需要新写 BoxCrop 类、把离线分块脚本改造成在线可调、新写独立的边界融合推理脚本，合计约 5 – 6 人/天的工程量。
4. 即使补完代码，S3DIS（约 10^6 点/Area，显存远低于硬件上限）也不是验证“切块参数”的合适数据集——这套参数的设计目标场景是 ArCH/SensatUrban/DALES 这类真正超出显存的大场景。

建议

✓ 两周冲刺阶段（当前进度）：继续使用已验证可执行的 v2 四因子方案（grid_size / point_max / jitter_sigma / dropout_ratio），把统计设计的严谨性（PB 筛选 → BBD 建模 → 回归/显存模型）用在能立刻产出数据的参数上。

✓ 论文后续阶段（任务书第五 – 八阶段）：在 ArCH / DALES 等真正大场景上，投入工程时间实现 spatial_shape / step / align_mode 三个缺失组件，把任务书原始的 7 因子设计用在它本来要解决的问题上，数据才有意义。